AI23531 DEEP LEARNING LAB

Experiment 10: Image Generation using Generative Adversarial Network (GAN)

AIM

To generate synthetic images using a GAN and inspect the generated samples.

ALGORITHM

1. Load and normalize the MNIST dataset.
2. Build a generator that maps random noise to an image.
3. Build a discriminator that distinguishes real images from generated images.
4. Train the discriminator on real and fake images, then train the generator to fool the discriminator.
5. Generate and display images from random noise.

PROGRAM

```
import torch
import torch.nn as nn
import matplotlib.pyplot as plt
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

device = "cuda" if torch.cuda.is_available() else "cpu"
loader = DataLoader(datasets.MNIST("./data", train=True, download=True,
    transform=transforms.Compose([transforms.ToTensor(),
        transforms.Normalize((0.5,), (0.5,))])),
    batch_size=128, shuffle=True)

G = nn.Sequential(nn.Linear(100, 256), nn.ReLU(),
    nn.Linear(256, 512), nn.ReLU(), nn.Linear(512, 784), nn.Tanh()).to(device)
D = nn.Sequential(nn.Linear(784, 512), nn.LeakyReLU(0.2),
    nn.Linear(512, 256), nn.LeakyReLU(0.2),
    nn.Linear(256, 1), nn.Sigmoid()).to(device)
optG = torch.optim.Adam(G.parameters(), lr=0.0002)
optD = torch.optim.Adam(D.parameters(), lr=0.0002)
criterion = nn.BCELoss()

for epoch in range(3):
    for real, _ in loader:
        real = real.to(device).flatten(1); n = len(real)
        ones = torch.ones(n, 1, device=device); zeros = torch.zeros(n, 1, device=device)
        noise = torch.randn(n, 100, device=device)
        fake = G(noise).detach()
        lossD = criterion(D(real), ones) + criterion(D(fake), zeros)
        optD.zero_grad(); lossD.backward(); optD.step()
        noise = torch.randn(n, 100, device=device)
        lossG = criterion(D(G(noise)), ones)
        optG.zero_grad(); lossG.backward(); optG.step()
    print("Epoch", epoch+1, "D loss:", round(lossD.item(), 3),
          "G loss:", round(lossG.item(), 3))

with torch.no_grad():
    imgs = G(torch.randn(16, 100, device=device)).cpu().reshape(-1, 28, 28)
fig, axes = plt.subplots(2, 8, figsize=(8, 3))
for ax, img in zip(axes.flat, imgs):
    ax.imshow(img, cmap="gray"); ax.axis("off")
plt.show()
```

RESULT

The GAN trains a generator and discriminator adversarially and displays synthetic digit-like images. Visual quality depends on training duration and hardware.